

Tentu, mari kita lanjut ke **Prewitt.cs**.

Jika Canny adalah algoritma yang "cerdas dan kompleks", maka **Prewitt** adalah algoritma yang "sederhana dan klasik". Kode ini jauh lebih pendek daripada Canny, tetapi prinsip dasarnya sangat penting karena menjadi fondasi bagi deteksi tepi lainnya.

Berikut adalah bedah kodenya untuk persiapan UAS.

1. Gambaran Besar: Apa itu Prewitt?

Prewitt adalah metode deteksi tepi yang menggunakan matematika matriks sederhana (Konvolusi) untuk melihat perubahan warna.

- Dia punya dua "kacamata" (Kernel): satu untuk melihat garis tegak (Vertikal), satu untuk garis datar (Horizontal).
- Hasil akhirnya adalah gabungan dari kedua penglihatan tersebut.

2. Penjelasan Kode Per Bagian

A. Persiapan Memori (LockBits)

Sama seperti Canny, bagian ini wajib ada agar pemrosesan gambar tidak lambat.

C#

```
public Bitmap Apply(Bitmap source)
{
    // 1. Setup LockBits (Akses Memori Cepat)
    Bitmap bmp = new Bitmap(source);

    // Mengunci gambar di memori agar bisa diakses sebagai array byte[]
    BitmapData srcData = bmp.LockBits(..., ImageLockMode.ReadOnly,
    PixelFormat.Format32bppArgb);

    // Menyiapkan dua wadah:
    // pixelBuffer = wadah data gambar asli (untuk dibaca)
    // resultBuffer = wadah data gambar hasil (untuk ditulis)
    byte[] pixelBuffer = new byte[bytes];
    byte[] resultBuffer = new byte[bytes];
```

```
Marshal.Copy(srcData.Scan0, pixelBuffer, 0, bytes); // Salin data gambar ke wadah
```

Poin UAS:

- Jelaskan bahwa kita butuh **dua buffer** (wadah). Satu untuk dibaca (pixelBuffer), satu untuk ditulis (resultBuffer). Kita tidak boleh menulis langsung ke wadah yang sedang dibaca karena akan mengacaukan perhitungan pixel sebelahnya.

B. Definisi Kernel (Inti Algoritma)

Inilah ciri khas Prewitt. Matriks 3x3 yang berisi angka -1, 0, 1.

C#

```
// Gx = Deteksi Garis Vertikal (Perubahan Kiri ke Kanan)
```

```
int[,] gx = {  
    { -1, 0, 1 },  
    { -1, 0, 1 },  
    { -1, 0, 1 }  
};
```

```
// Gy = Deteksi Garis Horizontal (Perubahan Atas ke Bawah)
```

```
int[,] gy = {  
    { -1, -1, -1 },  
    { 0, 0, 0 },  
    { 1, 1, 1 }  
};
```

Perbedaan dengan Sobel:

- **Prewitt:** Angkanya rata [-1, 0, 1]. Dia memperlakukan semua pixel tetangga sama pentingnya.
- **Sobel:** Angkanya [-1, 0, 1] tapi yang tengah [-2, 0, 2]. Sobel memberikan "bobot lebih" pada pixel yang paling dekat (tengah), sehingga hasilnya sedikit lebih halus daripada Prewitt.

C. Proses Konvolusi (Loop Parallel)

Bagian ini adalah mesin utamanya.

C#

```
// Parallel.For digunakan agar semua otak CPU bekerja bersamaan
Parallel.For(1, height - 1, y =>
{
    for (int x = 1; x < width - 1; x++)
    {
        double sumX = 0; // Penampung total nilai vertikal
        double sumY = 0; // Penampung total nilai horizontal

        // Konvolusi 3x3: Loop kecil mengelilingi pixel pusat
        for (int ky = -1; ky <= 1; ky++)
        {
            for (int kx = -1; kx <= 1; kx++)
            {
                // Ambil nilai pixel tetangga
                int pos = (y + ky) * stride + (x + kx) * 4;
                int val = pixelBuffer[pos]; // Ambil salah satu channel (Blue)

                // Kalikan dengan Kernel
                sumX += val * gx[ky + 1, kx + 1];
                sumY += val * gy[ky + 1, kx + 1];
            }
        }

        // ... (lanjut ke perhitungan magnitude)
    }
}
```

D. Menghitung Kekuatan Garis (Magnitude)

Setelah kita tahu seberapa kuat garis vertikal (sumX) dan horizontal (sumY), kita gabungkan.

C#

```
// Rumus Pythagoras: a^2 + b^2 = c^2
// Ini untuk mencari panjang sisi miring (Kekuatan total tepi)
```

```
int magnitude = (int)Math.Sqrt((sumX * sumX) + (sumY * sumY));
```

```
// Clamping (Pencegahan Error)
```

```
// Hasil matematika bisa saja > 255 atau < 0.
```

```
// Kita harus paksa agar tetap masuk akal untuk warna (0-255).
```

```
if (magnitude > 255) magnitude = 255;
```

```
if (magnitude < 0) magnitude = 0;
```

```
// Simpan ke Buffer Hasil (Grayscale)
```

```
// R, G, dan B diisi angka yang sama agar jadi abu-abu/putih.
```

```
int k = y * stride + x * 4;
```

```
resultBuffer[k] = (byte)magnitude; // Blue
```

```
resultBuffer[k + 1] = (byte)magnitude; // Green
```

```
resultBuffer[k + 2] = (byte)magnitude; // Red
```

```
resultBuffer[k + 3] = 255; // Alpha (Tidak Transparan)
```

```
}
```

```
});
```

Simulasi Pertanyaan Sidang/UAS untuk Prewitt.cs

1. Pertanyaan Perbandingan (Paling Sering Muncul)

Dosen: "Apa bedanya Prewitt dengan Sobel? Kapan harus pakai Prewitt?"

Jawaban Anda:

"Bedanya ada di **Kernel (Matriksnya)**, Pak.

- **Sobel** menggunakan angka pembobot 2 di tengah, sehingga lebih mementingkan pixel terdekat. Hasilnya lebih tahan terhadap noise (bintik-bintik).
- **Prewitt** menggunakan angka rata 1, sehingga lebih sensitif terhadap noise tetapi sedikit lebih cepat dihitung karena angkanya sederhana.

Kita pakai Prewitt kalau ingin komputasi yang paling ringan dan noise pada gambar sedikit."

2. Pertanyaan tentang Grayscale

Dosen: "Di kode ini kamu mengambil pixelBuffer[pos] langsung, padahal formatnya ARGB. Itu channel warna apa yang diambil? Kenapa bisa begitu?"

Jawaban Anda:

"Dalam format memori Windows (Little Endian), urutan byte-nya adalah B-G-R-A. Jadi pixelBuffer[pos] itu mengambil channel **Blue**.

Namun, algoritma deteksi tepi biasanya diasumsikan bekerja pada gambar

Grayscale (Abu-abu). Pada gambar grayscale, nilai $R = G = B$. Jadi mengambil Blue saja sudah cukup mewakili kecerahan pixel tersebut."

3. Pertanyaan tentang Parallel.For

Dosen: "Kenapa loop luarnya pakai Parallel.For, tapi loop dalamnya (for int x...) pakai for biasa?"

Jawaban Anda:

"Karena overhead (biaya memecah tugas) itu mahal, Pak.

Kalau kita memecah tugas terlalu kecil (per pixel), komputer malah sibuk membagi tugas daripada mengerjakannya.

Paling efisien adalah membagi tugas **per baris (per y)**. Jadi setiap inti CPU (Core) mendapat jatah memproses beberapa baris gambar sekaligus."

4. Pertanyaan tentang Clamping

Dosen: "Apa gunanya baris if (magnitude > 255) magnitude = 255;? Memangnya hasil Math.Sqrt bisa lebih dari 255?"

Jawaban Anda:

"Bisa, Pak.

Nilai maksimal Kernel Prewitt itu penjumlahan absolutnya adalah 3 (1+1+1).

Jika pixel tetangga sangat kontras (misal 0 bersebelahan dengan 255), hasil konvolusi (sumX) bisa mencapai sekitar 765 (3 * 255).

Akar kuadrat dari (765² + 765²) itu sekitar 1081.

Karena tipe data byte hanya muat sampai 255, kita harus memotong (clamp) nilai berlebih itu jadi 255 (Mentok Putih)."

Apakah penjelasan Prewitt ini sudah cukup jelas, atau mau lanjut ke file lain (jika ada)?